

PRESENTATO DA

Alfonso Mammato

DATA

14 Giugno 2026

STREAMLIT: IL FRAMEWORK PER IL NOSTRO CHATBOT

Interfaccia chatbot in poche righe di Python • app_base.py: meno di 30 righe di codice

```
import streamlit as st
import anthropic
```

```
st.title("🤖 Chatbot WiData")
if "messages" not in st.session_state:
    st.session_state.messages = []
```

```
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])
```

```
if prompt := st.chat_input("Scrivi..."):
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("assistant"):
        risposta = "Ciao, sono un assistente IoT!"
        st.markdown(risposta)
    st.session_state.messages.append({"role": "assistant", "content": risposta})
```

- Lo script viene rieseguito dall'inizio ad ogni interazione
- Nessun ciclo di eventi come React o Flask, nessun callback esplicito
- Ogni click, messaggio o slider → riparte dalla riga 1
- Paradigma dichiarativo: dichiarare la UI, Streamlit applica le diff
- Simile a React con virtual DOM, ma in Python e con molta meno cerimonia
- Conseguenza: tutto ciò che deve sopravvivere va nel session state

● Il Problema

Senza memoria, la cronologia chat sparirebbe ad ogni rerun. La lista messages verrebbe ricreata vuota ogni volta.

● Il Flusso

Utente invia prompt → aggiungi a messages. Ricevi risposta → aggiungi anche quella. Al prossimo rerun, il ciclo for mostra tutto lo storico.

● Errore Comune

Dimenticare la guardia if: la variabile viene sovrascritta ad ogni rerun e i dati si perdono immediatamente.

● La Soluzione

st.session_state è un dizionario magico che persiste tra i rerun. Pattern: if "messages" not in st.session_state.

● Elementi Persistenti

In app_rag.py: messages (cronologia), collection (database Chroma), token_totali (costi).
"Nuova chat" resetta + st.rerun().



Senza session state → nessuna memoria. Con session state → interfacce complesse e reattive per chatbot professionali.

COMPONENTI CHIAVE PER IL CHATBOT

Widget Base (app_base.py)

st.chat_input

Casella di testo specializzata per chat, restituisce il contenuto quando si preme Invio

st.chat_message

Crea una bolla con il ruolo (user/assistant) e un'icona

POTENZA DEL RERUN



Ogni movimento dello slider → lo script riparte → legge il nuovo valore → la risposta sarà più o meno "creativa"

Widget Avanzati (app_avanzata.py)

st.file_uploader – carica PDF, processati con PyPDF

st.spinner – feedback visivo durante operazioni lunghe

st.slider – controlla temperature e max_tokens

st.metric – mostra token usati e costo stimato

st.feedback – thumbs up/down per valutazioni

@ST.CACHE_RESOURCE: CRUCIALE PER RAG

● Il Problema

- In app_rag.py inizializziamo ChromaDB e la funzione di embedding
- Senza cache: a ogni rerun si perderebbe tutto (chunk indicizzati) e si dovrebbe ricaricare il PDF
- Sarebbe lentissimo e impraticabile

● La Soluzione

- **@st.cache_resource** sopra get_chroma_client()
- Streamlit crea il client **una sola volta**
- Riutilizzato per tutti i rerun successivi
- I dati rimangono al suo interno
- Vale anche per la funzione di embedding

Streaming: UI Professionale e Reattiva

● Pattern Corretto

placeholder = st.empty() → risposta = "" →
nel ciclo: placeholder.markdown(risposta +
"") → alla fine:
placeholder.markdown(risposta)

● Cosa NON Fare

Mai usare st.write() dentro il ciclo di
streaming: ogni chiamata crea un nuovo
elemento, la pagina "salta" e l'esperienza
utente viene rovinata.

● Vantaggi UX

Feedback immediato con cursore (typing).
Latenza percepita ridotta. st.expander
mostra i chunk RAG utilizzati → trasparenza
e fiducia nell'AI.



Want to make a presentation like this one?

Start with a fully customizable template, create a beautiful deck in minutes, then easily share it with anyone.

Create a presentation (It's free)